Bases de Data Engineering et Machine Learning

Séance 2 : Ingestion automatisée avec Python

Bac+4 Data, Institut GoldenCollar | Mouaad AGOURRAM

Déroulé de la séance

1. Théorie

Usage métier, contrat de données, architecture d'ingestion, état, reprise et observabilité.

2. Démonstration

Notebook Jupyter : observer un échec, corriger la donnée et vérifier le rejeu.

3. TP

CSV, API JSON et PostgreSQL : appliquer les contrôles et produire les preuves d'exécution.

Trois usages, trois architectures

LinkedIn

Tableau de bord quotidien : disponibilité, fraîcheur et détection du retard.

Uber

Données transactionnelles : ingestion incrémentale pour éviter le recalcul complet.

BAAC

Publication annuelle : fichier brut conservé, schéma explicite et rejeu reproductible.

Cadrer le workload d'ingestion

Caractéristiques du workload

Sources	Fichiers, API, bases opérationnelles, journaux d'événements ou SaaS.
Volume et throughput	Taille initiale, croissance, débit moyen et pics d'arrivée.
Latence	Batch, micro-batch ou streaming selon la fraîcheur réellement utile.
Accès et variabilité	Quotas, fenêtres de charge, évolution du schéma et fréquence des changements.

SLO du produit de données

08:00

heure limite de publication

Freshness	Données de J-1 disponibles avant l'ouverture.
Completeness	Toutes les partitions et au moins 99,9 % du volume attendu.
Correctness	Schéma compatible et contrôles critiques passés.
Réaction	Publication bloquée, alerte envoyée et dernière version certifiée conservée.

Évaluation technique de la source

Contrat	Format, schéma, clés, encodage, valeurs manquantes et compatibilité.
Accès	Identité, réseau, secrets, quotas, pagination et fenêtres autorisées.
Échelle	Historique, volume quotidien, fréquence de changement et croissance.
Reprise	Rétention, backfill, doublons, absence de source et données tardives.

Data contract et responsabilités

Incident : la source renomme customer_id sans préavis

Producteur — Versionne le schéma, explique la sémantique et annonce la rupture.

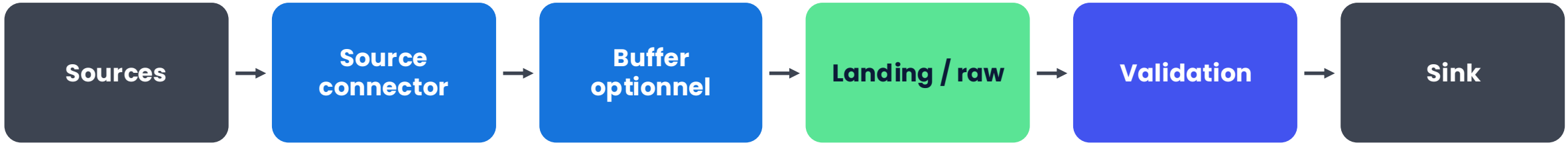
Équipe data — Détecte l'incompatibilité, bloque la publication et prépare la reprise.

Consommateur — Déclare les champs critiques et valide la nouvelle version du produit.

Sécurité : droits minimaux, rétention et audit sur toute la chaîne.

Architecture et exploitation d'un pipeline d'ingestion

Architecture logique du pipeline



Chemin d'erreur
Retry, quarantine ou dead-letter queue

Control plane
Orchestration, état, secrets, catalogue, lineage et métriques

Le buffer et la DLQ ne sont pas obligatoires : ils répondent à un besoin de débit ou de reprise.

Batch, micro-batch ou streaming

Critère	Batch	Micro-batch	Streaming
Données	Ensemble borné	Nouveaux éléments par fenêtre	Flux non borné
Latence	Minutes à jours	Secondes à minutes	Subseconde à secondes
État	Run et partitions	Checkpoint	Offset, checkpoint, watermark
Complexité	Faible	Intermédiaire	Élevée si traitement stateful

Full load, incremental load et CDC

Full load

Petit périmètre ou reconstruction complète. Aucun état de progression nécessaire.

Incremental load

High-water mark sur une date ou un identifiant. Attention aux retards et aux suppressions.

CDC

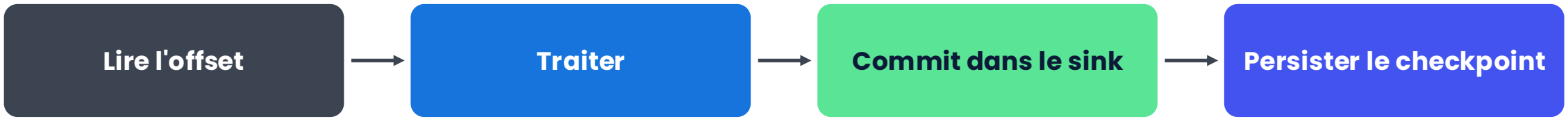
Initial snapshot puis lecture du transaction log avec offset ou LSN persistant.

Checkpoint, offset et watermark

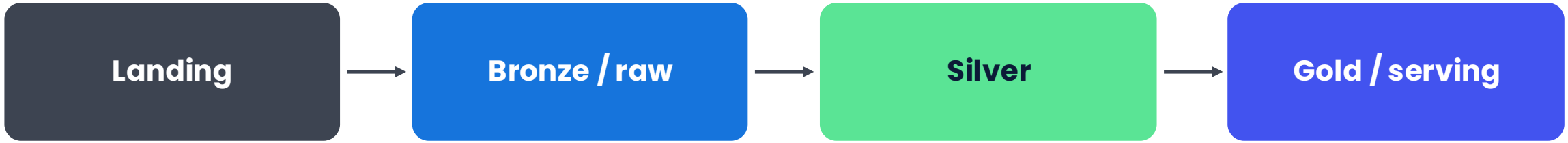
Offset — Position lue dans un log, une partition ou un journal transactionnel.

Checkpoint — État durable utilisé pour reprendre le pipeline après une interruption.

Watermark — Estimation de progression en event time pour les fenêtres et les late events.



Landing, bronze, silver et gold



Landing — Objet reçu, horodatage, source et empreinte. Zone de réception.

Bronze — Historique brut et fidèle, conservé pour le replay et l'audit.

Silver et gold — Validation puis publication d'un produit orienté usage.

Tolérance aux pannes et garanties de livraison



Erreur transitoire

Retry avec délai et nombre de tentatives limité

Erreur persistante

Quarantine ou dead-letter queue, puis replay contrôlé

At-least-once + clé stable + upsert idempotent = reprise sans duplication du résultat.

Observabilité du pipeline et de la donnée

RUN 2026-10-28T07:31

07:43

publication réussie

48 612 lignes

offset 8 412 995

Pipeline

Durée, throughput, erreurs, retries et état de l'orchestrateur.

Data

Freshness, completeness, volume, doublons et schema drift.

Traçabilité

run_id, version du code, source, checkpoint, cible et data lineage.

Alerting

Seuils liés au SLO et comparaison avec le comportement habituel.

Sécurité du pipeline d'ingestion

Identity	Service account ou managed identity dédiée au pipeline.
Least privilege	Lecture sur la source, écriture limitée au sink et droits régulièrement revus.
Secrets et réseau	Secret manager, chiffrement TLS et accès privé lorsque nécessaire.
Minimisation	Colonnes, historique et rétention alignés sur la finalité déclarée.

Preuves attendues : politique IAM, rotation des secrets, audit trail et règles de rétention.

Interfaces de source et choix d'outils

CSV : un contrat absent du fichier

```
Num_Acc ; dep ; lat ; long
202400000001 ; 01 ; 45,782 ; -1,234
```

Une lecture peut réussir tout en donnant des colonnes fausses.

Point-virgule — Délimiteur de champs déclaré explicitement.

Virgule — Séparateur décimal, pas délimiteur de colonnes.

Code 01 — Type texte pour conserver le zéro initial.

Contrôles — Schéma, volume, unicité et valeurs sentinelles.

API JSON : interface et dépendance

Contrat HTTP

Authentification
Timeout et retry
Rate limit
Pagination
Statuts et en-têtes

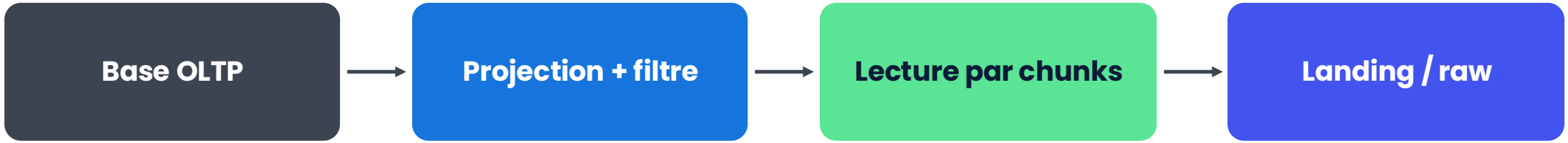
Contrat JSON

Schéma et types
Objets et listes imbriqués
Identifiants stables
Valeurs facultatives
Normalisation documentée

Preuve de collecte

Réponse brute, paramètres, instant de collecte et compte total attendu

Base opérationnelle : limiter l'impact



Source — Compte read-only, index adapté et fenêtre de charge maîtrisée.

Extraction — Colonnes utiles, prédicat incrémental et taille de chunk mesurée.

Isolation — Read replica ou CDC lorsque le workload justifie la complexité.

Choix du moteur : pandas ou Polars

pandas

Points forts

API familière, écosystème mature et intégration directe avec NumPy et scikit-learn.

Limites

Données généralement en mémoire. Les opérations successives peuvent créer des intermédiaires.

Polars

Points forts

Moteur multithreadé, expressions, optimisation lazy et lecture sélective des données.

Limites

API d'expressions à apprendre. Écosystème plus restreint et conversions parfois nécessaires.

Quand le batch manque son SLO

Objectif (SLO)

08:00

Mesure observée (SLI)

09:20

1. Profiler

Durée par étape, octets lus, lignes traitées et mémoire maximale.

2. Réduire

Filtrer plus tôt, projeter les colonnes et éviter le recalcul complet.

3. Vérifier

Même résultat métier, mêmes contrôles et exécution rejouable.

Décision

Changer de moteur seulement si le goulot reste démontré

SLI (Service Level Indicator) : mesure observée, ici la publication à 09:20.

SLO (Service Level Objective) : cible du SLI, ici une publication avant 08:00.

Du notebook au job orchestré



Notebook Observation interactive, hypothèses visibles et comparaison de plusieurs lectures.

Job Configuration externe, exécution sans état caché, logs, retries et alertes.

Le code de la démonstration reste dans Jupyter ; les slides portent les décisions et le vocabulaire.

Démo

Du contrat au pipeline idempotent

seance2/demo_ingestion.ipynb

Durée : 35 min

Parcours de la démonstration

1. Observer	Identifier le contrat réel et les champs dont le type porte du sens.
2. Faire échouer	Voir le contrôle bloquer la publication et produire un diagnostic.
3. Corriger	Publier une sortie validée avec sa preuve d'exécution.
4. Rejouer	Relancer sans doublon, puis tester high-water mark et late arrival.

À regarder pendant la démo : types, contrôles, run metadata, nombre de lignes et état incrémental.

Mise en pratique

TP 1 · 35 min	CSV BAAC : contrat de lecture et contrôles d'import.
TP 2 · 25 min	API Géo : preuve brute, normalisation JSON et couverture de jointure.
TP 3 · 35 min	PostgreSQL : full load, incremental load et reprise.
TP 4 · autonomie	Extraction des fonctions et passage à un job paramétré.

TP 1

Contrat CSV et contrôles d'import

seance2/tp/TP1_csv_enonce.ipynb

Durée : 35 min

TP 2

API JSON et jointure géographique

seance2/tp/TP2_json_enonce.ipynb

Durée : 25 min

TP 3

PostgreSQL complet et incrémental

seance2/tp/tp3_postgres/

Durée : 35 min

Avant la séance 3

Définition de terminé

SLO	Usage, freshness et réaction attendue en cas d'échec.
Raw	Source brute intacte, horodatée et identifiable.
Contrat	Schéma, types et contrôles bloquants documentés.
Replay	Commande reproductible et procédure décrite dans le README.

Principes à retenir

Workload et SLO	Ils déterminent le mode de traitement et le niveau de complexité utile.
Raw et état	Ils rendent possibles le replay, le backfill et l'audit.
Idempotence	At-least-once devient maîtrisable avec une clé stable et un upsert.
Observabilité	Le pipeline et le produit de données possèdent des signaux distincts.

Quiz de la séance 2

Question 1

Une direction accepte des chiffres chaque matin. Quel mode de traitement convient d'abord ?

A. Flux milliseconde

B. Lot certifié avant ouverture

C. Chargement complet chaque minute

D. Aucun contrôle

Question 2

Quand faut-il persister le checkpoint incrémental ?

A. Avant la lecture

B. Après la lecture

C. Après publication réussie

D. À chaque ligne

Question 3

Pourquoi conserver la réponse brute d'une API ?

A. Pour éviter les types

B. Pour rejouer sans rappeler la source

C. Pour supprimer les contrôles

D. Pour accélérer le réseau

Question 4

Quel signal mesure le mieux la fraîcheur ?

A. Nombre de colonnes

B. Retard entre source et publication

C. Taille du script

D. Nombre de notebooks

Question 5

Comment absorber un événement tardif sans doublon ?

A. Ignorer les retards

B. Relire une fenêtre et faire un upsert

C. Avancer le watermark

D. Recharger sans clé

Merci ! Des questions ?

